

Report

- 1) Code file Location: https://universityofsussex-my.sharepoint.com/:f/g/personal/ab2429_sussex_ac_uk/lgBRt5Yes_BnRYZe3h5GNpw4AS3WqD4f5M8ZwGz4f6W2b3A?e=4YX3hg
- 2) Panopto recording: <https://sussex.cloud.panopto.eu/Panopto/Pages/Viewer.aspx?id=cea0af66-4508-41f9-b4f9-b3b000545b56>

Task 0 – Self-reflection

The **marking grid** should be completed **objectively and accurately** to reflect the standard of your work. Indicate all **suggested features** that have been successfully implemented by marking the corresponding boxes. Any **additional security features** you have developed should be listed in the **'Suggestive Features'** column where placeholders (dashes) are provided. Finally, **identify and highlight** the **appropriate level of attainment** (*Poor, Average to Good, or Excellent*) that most accurately represents the quality and completeness of your submission.

Excellent [7 to 10 marks] Implementation has no flaws, and the student has gone beyond to be considered for this. [You must mention in self-reflection if you have implemented more features]	Average to good [4 to 7 marks]	Poor or evidence is not clear in the report. [0 to 4 marks]	Tick (Y) / Cross (X)	Suggestive features to implement	Criteria
---	-----------------------------------	---	-------------------------------	-------------------------------------	----------

Excellent [7 to 10 marks] Implementation has no flaws, and the student has gone beyond to be considered for this. [You must mention in self-reflection if you have implemented more features]	Average to good [4 to 7 marks]	Poor or evidence is not clear in the report. [0 to 4 marks]	Tick (Y) / Cross (X)	Suggestive features to implement	Criteria
Policy has no flaw, and its implementation is excellent. Various mechanisms implemented to ensure password policy is secure. 9	Policy has no flaws, but implementation of policy is simple.	Policy has many flaws for example password is not encrypted, and no salt applied. Password forgot policy has security flaws.	Y	Password entropy	Password policy- 10marks
			Y	Security questions	
			Y	Password recovery	
			Y	Account Lockout	

Several countermeasures are implemented, and the quality of countermeasures are excellent. Other vulnerabilities are dealt with. 9	Countermeasures are implemented in all the pages however quality of implementation is simple.	Very little effort to implement countermeasures to avoid these vulnerabilities.	Y	SQL injection	Vulnerabilities-10 marks
			Y	XSS	
			Y	CSR	
			Y	File Upload	

Excellent implementation. Student has gone beyond.	All requirements are implemented to authenticate the user. However, quality of implementation is simple. 7	Lots of obvious authentication's requirements are not implemented.	Y	Email verification for registration,	Authentication – 10 marks
			X	2 factor authentications (PIN)	
			Y	2 factor authentications (email)	

Excellent implementation of countermeasures against these attacks.	No flaws in countermeasures however quality of implementation	Very little effort against these attacks	Y	Brute force attack – Number	Obfuscation/Common attacks – 10 marks

8	is simple.			of attempts	
			Y	Botnet attack – Captcha	
			Y	Dictionary attack/Rainbow table attack	

28			Y	User registration	Features of web application – 30 marks
			Y	User login	
			Y	Forgot password	
			Y	Evaluation	
			Y	List evaluation	
			Y	Database design and its security	
15			Y		Access control -15 marks
7			Y	All the marking criteria covered	Video – 9 marks
6			Y	Completion of this marking grid clearly	Self-reflection – 6 marks

Part A – Access Control

Task 1 – Setup Environment

- Created a working directory named `computerSecurity_lab` in my home folder using the Linux terminal.
- Inside this directory, created three files: `file1.txt`, `file2.sh`, and `secret.doc`.
- Listed the contents of the directory with detailed information to show the default permissions for each file (owner, group and others).

```
root@LAPTOP-TALOA354:~# mkdir computerSecurity_lab
root@LAPTOP-TALOA354:~# cd computerSecurity_lab
root@LAPTOP-TALOA354:~/computerSecurity_lab# touch file1.txt file2.sh secret.doc
root@LAPTOP-TALOA354:~/computerSecurity_lab# ls -l
total 0
-rw-r--r-- 1 root root 0 Dec  6 21:17 file1.txt
-rw-r--r-- 1 root root 0 Dec  6 21:17 file2.sh
-rw-r--r-- 1 root root 0 Dec  6 21:17 secret.doc
root@LAPTOP-TALOA354:~/computerSecurity_lab#
```

Figure A1: Creation of `computerSecurity_lab` directory and default file permissions.

Task 2 – Modify Permissions

- Updated the permissions on `file1.txt` so that only the owner can read and write it, while the group and others have no permissions.
- Changed `file2.sh` to be executable by all users, allowing it to be run as a script.
- Set `secret.doc` to be read-only for every user (owner, group and others).

- Displayed the updated permissions for all three files to confirm that the changes were applied correctly.

```
root@LAPTOP-TALOA354:~/computerSecurity_lab# chmod 600 file1.txt
root@LAPTOP-TALOA354:~/computerSecurity_lab# chmod 755 file2.sh
root@LAPTOP-TALOA354:~/computerSecurity_lab# chmod 444 secret.doc
root@LAPTOP-TALOA354:~/computerSecurity_lab# ls -l
total 0
-rw----- 1 root root 0 Dec  6 21:17 file1.txt
-rwxr-xr-x 1 root root 0 Dec  6 21:17 file2.sh
-r--r--r-- 1 root root 0 Dec  6 21:17 secret.doc
root@LAPTOP-TALOA354:~/computerSecurity_lab#
```

Figure A2: Updated permissions for file1.txt, file2.sh and secret.doc.

Task 3 – Change Ownership

- Created a new user account named bob on the system (where permitted).
- Changed the ownership of file1.txt so that bob becomes the file owner while keeping the same group.
- Created a new group named ethicalHackers and changed the group ownership of the computerSecurity_lab directory to this group.
- Verified the new owner and group values using a detailed directory listing.

```
root@LAPTOP-TALOA354:~/computerSecurity_lab# cd ..
root@LAPTOP-TALOA354:~# useradd -m Bob
root@LAPTOP-TALOA354:~# groupadd ethicalHackers
root@LAPTOP-TALOA354:~# chown Bob computerSecurity_lab/file1.txt
root@LAPTOP-TALOA354:~# chgrp ethicalHackers computerSecurity_lab
Command 'chgrp' not found, did you mean:
  command 'chr' from snap chr (0.80.4)
  command 'chrt' from deb util-linux (2.37.2-4ubuntu3.4)
  command 'chgrp' from deb coreutils (8.32-4.1ubuntu1.2)
See 'snap info <snapname>' for additional versions.
root@LAPTOP-TALOA354:~# chgrp ethicalHackers computerSecurity_lab
root@LAPTOP-TALOA354:~# ls -l computerSecurity_lab/
total 0
-rw----- 1 Bob  root 0 Dec  6 21:17 file1.txt
-rwxr-xr-x 1 root root 0 Dec  6 21:17 file2.sh
-r--r--r-- 1 root root 0 Dec  6 21:17 secret.doc
root@LAPTOP-TALOA354:~# ls -ld computerSecurity_lab/
drwxr-xr-x 2 root ethicalHackers 4096 Dec  6 21:17 computerSecurity_lab/
root@LAPTOP-TALOA354:~#
```

Figure A3: Ownership of file1.txt changed to user bob and directory group set to ethicalHackers.

Task 4 – Directory Access Control

- Created a subdirectory named `shared_docs` inside the `computerSecurity_lab` directory.
- Set the permissions on `shared_docs` so that the owner has full access (read, write and execute), the group can read and execute, and others have no access at all.
- Used a detailed directory listing command to confirm that the permissions are correctly set to `rwxr-x---` for the `shared_docs` directory.

```
root@LAPTOP-TALOA354:~# cd computerSecurity_lab
root@LAPTOP-TALOA354:~/computerSecurity_lab# mkdir shared_docs
root@LAPTOP-TALOA354:~/computerSecurity_lab# chmod 750 shared_docs
root@LAPTOP-TALOA354:~/computerSecurity_lab# ls -ld shared_docs
drwxr-x--- 2 root root 4096 Dec  6 21:28 shared_docs
```

Figure A4: Access control configuration for the `shared_docs` subdirectory.

Task 5 – Umask Configuration

- Displayed the current umask value of the shell to understand the default permissions for newly created files.
- Changed the umask so that new files are created with permissions `rw-r-----` (owner: read/write, group: read-only, others: no access), i.e. using a umask value of `027`.
- Created a new test file in the working directory and listed its permissions to confirm that the new umask setting is applied

correctly.

```
root@LAPTOP-TALOA354: ~/computerSecurity_lab# umask
0022
root@LAPTOP-TALOA354: ~/computerSecurity_lab# umask 027
root@LAPTOP-TALOA354: ~/computerSecurity_lab# umask
0027
root@LAPTOP-TALOA354: ~/computerSecurity_lab# touch test_umask.txt
root@LAPTOP-TALOA354: ~/computerSecurity_lab# ls -l test_umask.txt
-rw-r----- 1 root root 0 Dec  6 21:30 test_umask.txt
root@LAPTOP-TALOA354: ~/computerSecurity_lab#
```

Figure A5: Umask adjusted to 027 and verified using a newly created test file.

Part B –

Task 1: Registration

The registration form allows new customers to create an account by providing an email address, password, full name and UK mobile number.

All data is validated on the server and only securely processed information is stored in the database.

Why this registration form is secure

Required field validation:

```
// 1) Required fields validation
if ($fullname === '') {
    $errors['fullname'] = 'Fullname is required';
}
if ($email === '') {
    $errors['email'] = 'Email is required';
}
if ($password === '') {
    $errors['password'] = 'Password is required';
}
if ($phone === '') {
    $errors['phone'] = 'Phone is required';
}
if ($securityQuestion === '') {
    $errors['security_question'] = 'Security question is required';
}
if ($securityAnswer === '') {
    $errors['security_answer'] = 'Security answer is required';
} elseif (strlen($securityAnswer) < 3) {
    $errors['security_answer'] = 'Security answer must be at least 3 characters';
}
```

Email address format validation:

```
// 2) Email format validation
if ($email !== '' && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $errors['email'] = 'Invalid email format';
}
```

Password Strength Verification:

```
// 3) Password strength validation (minimum 8 characters, must contain letters and numbers)
if ($password !== '' && strlen($password) < 8) {
    $errors['password'] = 'Password must be at least 8 characters long';
} elseif ($password !== '' && !preg_match('/^(?=.*[A-Za-z])(?=.*\d){8,}$/', $password)) {
    $errors['password'] = 'Password must contain both letters and numbers';
}
```

UK telephone verification:

```
// 4) UK phone number format validation (11 digits starting with 07)
if ($phone != '' && !preg_match('/^07\d{9}$/', $phone)) {
    $errors['phone'] = 'Invalid phone format (must be 11 digits starting with 07)';
}
```

Email Duplicate Check:

```
// 5) Check if email already exists in database
if (empty($errors['email'])) {
    $stmt = $pdo->prepare("SELECT id FROM users WHERE email = ? LIMIT 1");
    $stmt->execute([$email]);
    $existing_user = $stmt->fetch();

    if ($existing_user) {
        $errors['email'] = "This email is already registered";
    }
}
```

CAPTCHA verification:

```
// 6) CAPTCHA validation
$captchaAnswer = isset($_POST['captcha_answer']) ? trim($_POST['captcha_answer']) : '';
if ($captchaAnswer === '') {
    $errors['captcha'] = 'Please solve the CAPTCHA';
} elseif (!validateCaptcha($captchaAnswer)) {
    $errors['captcha'] = 'Incorrect CAPTCHA answer. Please try again.';
    // Generate new CAPTCHA for retry
    generateCaptcha();
}
```

Password hashing + Token generation + Security question hashing:

```
// 7) If no errors, insert user data into database
if (empty($errors)) {
    $passwordHash = password_hash($password, PASSWORD_DEFAULT);

    // Generate verification token
    $verificationToken = bin2hex(random_bytes(32)); // 64 hex characters
    $verificationExpires = date('Y-m-d H:i:s', time() + (24 * 3600)); // 24 hours

    // Hash security answer (similar to password hashing)
    $securityAnswerHash = password_hash(strtolower(trim($securityAnswer)), PASSWORD_DEFAULT);
}
```

SQL prepared statements:

```
= $pdo->prepare(
    "INSERT INTO users (email, password_hash, full_name, phone, is_admin, email_verified, verification_token, verification_token_expires, security_question, security_answer_hash)
    VALUES (:email, :password_hash, :full_name, :phone, 0, 0, :verification_token, :verification_expires, :security_question, :security_answer_hash)"

->execute([
    'email' => $email,
    'password_hash' => $passwordHash,
    'full_name' => $fullName,
    'phone' => $phone,
    'verification_token' => $verificationToken,
    'verification_expires' => $verificationExpires,
    'security_question' => $securityQuestion,
    'security_answer_hash' => $securityAnswerHash,
```

Send verification email:

```
// Send verification email
$emailResult = sendEmailVerification($email, $fullname, $verificationToken);

if ($emailResult['success']) {
    $registrationSuccess = true;
    if ($emailResult['link']) {
        $verificationLink = $emailResult['link'];
    }
} else {
    // Email sending failed, but account is created
    // Store verification link for display
    $config = require __DIR__ . '/includes/mail_config.php';
    $verificationLink = $config['app_url'] . '/email_verify.php?token=' . urlencode($verificationToken);
    $registrationSuccess = true;
}
```

Cross-Site Request Forgery (CSRF) validation:

```
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    // CSRF Protection
    if (!verifyCSRFToken()) {
        $errors['csrf'] = 'Invalid security token. Please try again.';
    }
}
```

Task 2 - Login

The login feature allows existing users to access their accounts securely using their email address and password. It combines input validation, CAPTCHA, CSRF protection, account lockout and email-based two-factor authentication to protect against common attacks.

Why this login feature is secure

Account Lock + Two-Factor Authentication Configuration:

```
define( string $name , mixed $value [, bool $case_insensitive ]): bool = Lockout
define('LOCKOUT_DURATION', 15 * 60); // Lockout duration in seconds (15 minutes)

// =====
// 2FA Configuration
// =====
define('TWO_FA_CODE_LENGTH', 6); // 6-digit code
define('TWO_FA_EXPIRY', 10 * 60); // Code expires in 10 minutes
```

CSRF protection + input validation:

```
// CSRF Protection
if (!verifyCSRFToken()) {
    $errors['csrf'] = 'Invalid security token. Please try again.';
}

$email    = isset($_POST['email']) ? trim($_POST['email']) : '';
$password = isset($_POST['password']) ? trim($_POST['password']) : '';

// 1) Required fields validation
if ($email === '') {
    $errors['email'] = 'Email is required';
}
if ($password === '') {
    $errors['password'] = 'Password is required';
}

// 2) Email format validation
if ($email !== '' && !filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $errors['email'] = 'Invalid email format';
}
```

CAPTCHA verification logic:

```
// 3) CAPTCHA validation
$captchaAnswer = isset($_POST['captcha_answer']) ? trim($_POST['captcha_answer']) : '';
if ($captchaAnswer === '') {
    $errors['captcha'] = 'Please solve the CAPTCHA';
} elseif (!validateCaptcha($captchaAnswer)) {
    $errors['captcha'] = 'Incorrect CAPTCHA answer. Please try again.';
    // Generate new CAPTCHA for retry
    generateCaptcha();
}
```

PDO prepared statement query:

```
$stmt = $pdo->prepare(
    "SELECT id, email, password_hash, full_name, phone, is_admin, email_verified,
       failed_login_attempts, lockout_until, last_failed_login
    FROM users WHERE email = ? LIMIT 1"
);
$stmt->execute([$email]);
$user = $stmt->fetch();
```

checkAccountLockout():

```
function checkAccountLockout($user) {
    if (empty($user['lockout_until'])) {
        return ['locked' => false, 'remaining_minutes' => 0];
    }

    $lockoutUntil = strtotime($user['lockout_until']);
    $now = time();

    if ($lockoutUntil > $now) {
        $remainingSeconds = $lockoutUntil - $now;
        $remainingMinutes = ceil($remainingSeconds / 60);
        return ['locked' => true, 'remaining_minutes' => $remainingMinutes];
    }

    return ['locked' => false, 'remaining_minutes' => 0];
}
```

```

$lockoutStatus = checkAccountLockout($user);

if ($lockoutStatus['locked']) {
    // Account is locked - show lockout message
    $lockoutMessage = "Account is temporarily locked due to too many failed login attempts. Please try again in {$lockoutStatus['remaining_minutes']} minute(s).";
    $errors['lockout'] = $lockoutMessage;
} else {

```

password_verify() + email_verified check:

```

if (password_verify($password, $user['password_hash'])) {
    // Check if email is verified
    if (!isset($user['email_verified']) || $user['email_verified'] == 0) {
        $errors['email'] = 'Please verify your email address before logging in. Check your email for the verification link.';
    }
}

```

recordFailedAttempt():

```

@return array ['locked' => bool, 'attempts_remaining' => int]
/

function recordFailedAttempt($pdo, $userId, $currentAttempts) {
    $newAttempts = $currentAttempts + 1;

    if ($newAttempts >= MAX_LOGIN_ATTEMPTS) {
        // Lock the account
        $lockoutUntil = date('Y-m-d H:i:s', time() + LOCKOUT_DURATION);
        $stmt = $pdo->prepare(
            'UPDATE users SET
             failed_login_attempts = :attempts,
             lockout_until = :lockout_until,
             last_failed_login = NOW()
            WHERE id = :id'
        );
        $stmt->execute([
            'attempts' => $newAttempts,
            'lockout_until' => $lockoutUntil,
            'id' => $userId
        ]);
        return ['locked' => true, 'attempts_remaining' => 0];
    }
}

```

```

$currentAttempts = (int)($user['failed_login_attempts'] ?? 0);
$result = recordFailedAttempt($pdo, $user['id'], $currentAttempts);

if ($result['locked']) {
    $lockoutMinutes = ceil(LOCKOUT_DURATION / 60);
    $lockoutMessage = "Too many failed login attempts. Your account has been locked for {$lockoutMinutes} minutes.";
    $errors['lockout'] = $lockoutMessage;
} else {
    // Show remaining attempts (but don't reveal user exists for security)
    $errors['password'] = "Invalid email or password. {$result['attempts_remaining']} attempt(s) remaining before account lockout.";
}

```

generate2FACode():

```

function generate2FACode($pdo, $userId) {
    // Generate random 6-digit code
    $code = str_pad(random_int(0, 999999), TWO_FA_CODE_LENGTH, '0', STR_PAD_LEFT);
    $expires = date('Y-m-d H:i:s', time() + TWO_FA_EXPIRY);

    // Store in database
    $stmt = $pdo->prepare(
        'UPDATE users SET two_fa_code = ?, two_fa_expires = ? WHERE id = ?'
    );
    $stmt->execute([$code, $expires, $userId]);

    return $code;
}

```

Generate 2FA + Send email + Redirect:

```
// Generate 2FA code
$twoFACode = generate2FACode($pdo, $user['id']);

// Send 2FA code via email
$emailResult = send2FACodeEmail($user['email'], $user['full_name'], $twoFACode);

// Store pending 2FA data in session
$_SESSION['pending_2fa_user_id'] = $user['id'];
$_SESSION['pending_2fa_email'] = $user['email'];
$_SESSION['pending_2fa_fullname'] = $user['full_name'];
$_SESSION['pending_2fa_phone'] = $user['phone'];
$_SESSION['pending_2fa_is_admin'] = $user['is_admin'];

// If email sending failed, store code for development testing
if (!$emailResult['success'] && isset($emailResult['code'])) {
    $_SESSION['dev_2fa_code'] = $emailResult['code'];
}

// Redirect to 2FA verification page
header('Location: verify_2fa.php');
exit;
```

Do not disclose whether the user exists:

```
// User doesn't exist - don't reveal this for security
// Just show generic error
$errors['email'] = 'Invalid email or password';
$errors['password'] = 'Invalid email or password';
}
```

Task 3 - Password Management

This feature provides password strength recommendations with real-time entropy calculation and feedback (weak/medium/strong/very strong).

Password recovery uses two-step verification: email address followed by security question verification. Reset tokens expire after 1 hour, and security answers are stored as bcrypt hashes for secure comparison.

Why this password management feature is secure

- Password 1: 123456Qwer@ --- Strong

- Password 2: 12345qwer@----- Medium

1.Password Recovery Request

Two-step verification flow-Step 1: email, Step 2: security question:

```
$step = isset($_SESSION['reset_step']) ? $_SESSION['reset_step'] : 1; // Step 1: email, Step 2: security question
$userEmail = isset($_SESSION['reset_email']) ? $_SESSION['reset_email'] : '';
```

CSRF Protection-verifyCSRFToken()

```
// CSRF Protection
if (!verifyCSRFToken()) {
    $errors['csrf'] = 'Invalid security token. Please try again.';
}
```

Email address format validation:

```
// Simple validation
if ($email === '') {
    $errors['email'] = 'Email is required';
} elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $errors['email'] = 'Invalid email format';
}
```

Obtain security question:

```
// Find user and get security question
$stmt = $pdo->prepare('SELECT id, email, full_name, security_question FROM users WHERE email = ? LIMIT 1');
$stmt->execute([$email]);
$user = $stmt->fetch(PDO::FETCH_ASSOC);
```

Secure password reset flow– verifies the security answer using password_verify() with case-insensitive comparison, then generates a secure 64-character reset token with a 1-hour expiry, stores it via a prepared statement and emails a password reset link containing the token:

```

if ($user && !empty($user['security_answer_hash'])) {
    // Verify security answer (case-insensitive)
    if (password_verify(strtolower(trim($securityAnswer)), $user['security_answer_hash'])) {
        // Answer is correct - generate reset token and send email
        $token = bin2hex(random_bytes(32)); // 64 hex characters
        $expires = date('Y-m-d H:i:s', time() + 3600);

        // Write to database
        $update = $pdo->prepare(
            'UPDATE users SET reset_token = :token, reset_expires = :expires WHERE id = :id'
        );
        $update->execute([
            'token' => $token,
            'expires' => $expires,
            'id' => $user['id'],
        ]);

        // Send password reset email
        $userName = $user['full_name'] ?? $user['email'];
        $emailResult = sendPasswordResetEmail($user['email'], $userName, $token);
    }
}

```

General Error Message - Do Not Reveal Whether the User Exists:

```

// User doesn't exist - don't reveal this, but show generic error
$errors['email'] = 'Invalid email or account not found.';

```

2.Set a new password

Verify that the token is valid and has not expired.:

```

function findUserByToken($pdo, $token) {
    $stmt = $pdo->prepare(
        'SELECT id, email, reset_expires
        FROM users
        WHERE reset_token = ?
        AND reset_expires IS NOT NULL
        AND reset_expires > NOW()
        LIMIT 1'
    );
    $stmt->execute([$token]);
    return $stmt->fetch(PDO::FETCH_ASSOC);
}

```

Cross-Site Request Forgery Protection:

```

// CSRF Protection
if (!verifyCSRFToken()) {
    $errors['csrf'] = 'Invalid security token. Please try again.';
}

```

Password strength validation:

```

} elseif (strlen($password) < 8) {
    $errors['password'] = 'Password must be at least 8 characters long';
} elseif (!preg_match('/^(?=.*[A-Za-z])(?=.*\d){8,}$/', $password)) {
    $errors['password'] = 'Password must contain both letters and numbers';
}

```

Password hashing:

```

if (empty($errors)) {
    $passwordHash = password_hash($password, PASSWORD_DEFAULT);
}

```

Clear reset token:

```

$stmt = $pdo->prepare(
    'UPDATE users
    SET password_hash = :hash,
        reset_token = NULL,
        reset_expires = NULL
    WHERE id = :id'
);

```

Password entropy calculation:

```

function calculatePasswordEntropy(password) {
    if (!password) return 0;
    | Ctrl+L to chat, Ctrl+K to generate
    let charsetSize = 0;
    let hasLower = /[a-z]/.test(password);
    let hasUpper = /[A-Z]/.test(password);
    let hasDigit = /[0-9]/.test(password);
    let hasSpecial = /[^\a-zA-Z0-9]/.test(password);

    if (hasLower) charsetSize += 26;
    if (hasUpper) charsetSize += 26;
    if (hasDigit) charsetSize += 10;
    if (hasSpecial) charsetSize += 32;

    if (charsetSize === 0) return 0;

    return Math.log2(Math.pow(charsetSize, password.length));
}

```

Password strength feedback

```
function checkPasswordStrength(password) {
  const strengthDiv = document.getElementById('password-strength');
  const entropyDiv = document.getElementById('password-entropy');

  if (!password) {
    strengthDiv.innerHTML = '';
    entropyDiv.innerHTML = '';
    return;
  }

  let strength = 0;
  let feedback = [];

  if (password.length >= 8) strength += 1; else feedback.push('At least 8 characters');
  if (/[a-z]/.test(password)) strength += 1; else feedback.push('Add lowercase letters');
  if (/[A-Z]/.test(password)) strength += 1; else feedback.push('Add uppercase letters');
  if (/[0-9]/.test(password)) strength += 1; else feedback.push('Add numbers');
  if (/[^a-zA-Z0-9]/.test(password)) strength += 1; else feedback.push('Add special characters');
  if (password.length >= 12) strength += 1;

  const entropy = calculatePasswordEntropy(password);

  let strengthClass = '';
  let strengthText = '';
  let strengthBarClass = '';

  if (strength <= 2) {
    strengthClass = 'strength-weak-text';
    strengthText = 'Weak';
    strengthBarClass = 'strength-weak';
  } else if (strength <= 4) {
    strengthClass = 'strength-medium-text';
    strengthText = 'Medium';
    strengthBarClass = 'strength-medium';
  } else if (strength <= 5) {
    strengthClass = 'strength-strong-text';
    strengthText = 'Strong';
  } else {
    strengthClass = 'strength-very-strong-text';
    strengthText = 'Very Strong';
    strengthBarClass = 'strength-very-strong';
  }

  strengthDiv.innerHTML = `
    <div class="password-strength-bar ${strengthBarClass}"></div>
    <div class="strength-text ${strengthClass}">Password Strength: ${strengthText}</div>
    ${feedback.length > 0 ? '<div style="font-size: 11px; color: #888; margin-top: 4px;">Suggestions: ' + feedback.slice(0, 2).join(', ') + '</div>' : ''}
  `;

  let entropyLevel = '';
  if (entropy < 28) entropyLevel = 'Low';
  else if (entropy < 35) entropyLevel = 'Medium';
  else if (entropy < 50) entropyLevel = 'High';
  else entropyLevel = 'Very High';

  entropyDiv.innerHTML = `Password Entropy: ${entropy.toFixed(1)} bits (${entropyLevel})`;
}
```

```

    <div class="password-strength-bar ${strengthBarClass}"></div>
    <div class="strength-text ${strengthClass}">Password Strength: ${strengthText}</div>
    ${feedback.length > 0 ? '<div style="font-size: 11px; color: #888; margin-top: 4px;">Suggestions: ' + feedback.slice(0, 2).join(', ') + '</div>' : ''}
  `;

  let entropyLevel = '';
  if (entropy < 28) entropyLevel = 'Low';
  else if (entropy < 35) entropyLevel = 'Medium';
  else if (entropy < 50) entropyLevel = 'High';
  else entropyLevel = 'Very High';

  entropyDiv.innerHTML = `Password Entropy: ${entropy.toFixed(1)} bits (${entropyLevel})`;

```

Task 4 Request Evaluation and File Upload

The “Request Evaluation” feature allows authenticated customers to submit evaluation requests for potential antique objects. The page provides a text area to describe the object, a dropdown box to choose the preferred contact method (phone or email) and a secure file upload field

for an optional photo of the object.

Why this request evaluation page is secure

1. Request Evaluation

1.1 Access Control:

```
1) Must be Logged in to access
(!isset($_SESSION['user_id'])) {
    header('Location: login.php');
    exit;
```

1.2 CSRF Protection:

```
CSRF Protection: Verify token on POST requests
($_SERVER['REQUEST_METHOD'] === 'POST') {
    if (!verifyCSRFToken()) {
        $_SESSION['flash_error'] = 'Invalid security token. Please try again.';
        header('Location: request_new.php');
        exit;
    }
```

1.3 Input Validation:

```
// 3) Basic validation
if ($subject === '') {
    $errors['subject'] = 'Subject is required';
}

if ($description === '') {
    $errors['description'] = 'Description is required';
}

// Optional: Add Length Limit
if ($description !== '' && strlen($description) < 20) {
    $errors['description'] = 'Please provide a bit more detail (at least 20 characters).';
}

// Validate contact preference
if (!in_array($contactPreference, ['phone', 'email'])) {
    $errors['contact_preference'] = 'Invalid contact preference selected.';
}
```

1.4 SQL Injection Prevention - Prepared Statements:

```
// 4) Insert into database
if (empty($errors)) {
    try {
        $stmt = $pdo->prepare(
            'INSERT INTO requests (user_id, subject, description, contact_preference, photo_path, status, created_at, updated_at)
            VALUES (:user_id, :subject, :description, :contact_preference, :photo_path, :status, NOW(), NOW())'
        );

        $stmt->execute([
            'user_id'      => $_SESSION['user_id'],
            'subject'      => $subject,
            'description'  => $description,
            'contact_preference' => $contactPreference,
            'photo_path'   => $photoPath, // Can be null if no photo uploaded
            'status'       => 'pending',
        ]);
    }
}
```

1.5 User Can Only View Own Requests:

```
// Regular user can only view their own requests
$stmt = $pdo->prepare(
    'SELECT r.id, r.subject, r.description, r.contact_preference, r.photo_path, r.status, r.created_at, r.updated_at,
    u.id as user_id, u.full_name as user_name, u.email as user_email, u.phone as user_phone
    FROM requests r
    LEFT JOIN users u ON r.user_id = u.id
    WHERE r.id = ? AND r.user_id = ?'
);
$stmt->execute([$requestId, $_SESSION['user_id']]);
```

2. File Upload

2.1 File Upload Configuration:

```
// Configuration for file uploads
define('UPLOAD_DIR', __DIR__ . '/uploads/');
define('MAX_FILE_SIZE', 5 * 1024 * 1024); // 5MB
define('ALLOWED_EXTENSIONS', ['jpg', 'jpeg', 'png', 'gif', 'webp']);
define('ALLOWED_MIME_TYPES', ['image/jpeg', 'image/png', 'image/gif', 'image/webp']);
```

2.2 File Size Validation:

```
// Check file size
if ($file['size'] > MAX_FILE_SIZE) {
    $errors[] = 'File size exceeds maximum allowed size (5MB).';
}
```

2.3 File Extension Whitelist Validation:

```
// Check file extension
$extension = strtolower(pathinfo($file['name'], PATHINFO_EXTENSION));
if (!in_array($extension, ALLOWED_EXTENSIONS)) {
    $errors[] = 'Invalid file type. Only JPG, PNG, GIF, and WEBP images are allowed.';
}
```

2.4 MIME Type Validation (using finfo):

```
// Check MIME type (more secure than extension check)
// Use finfo if available, otherwise fall back to mime_content_type or $_FILES['type']
if (function_exists('finfo_open')) {
    $finfo = finfo_open(FILEINFO_MIME_TYPE);
    $mimeType = finfo_file($finfo, $file['tmp_name']);
    finfo_close($finfo);
} elseif (function_exists('mime_content_type')) {
    $mimeType = mime_content_type($file['tmp_name']);
} else {
    // Fallback to $_FILES type (Less secure but better than nothing)
    $mimeType = $file['type'] ?? 'application/octet-stream';
}

if (!in_array($mimeType, ALLOWED_MIME_TYPES)) {
    $errors[] = 'Invalid file type detected (' . $mimeType . '). Only image files are allowed.'
}
```

2.5 Image Content Validation (getimagesize):

```
// Verify it's actually an image by checking image dimensions
$imageInfo = @getimagesize($file['tmp_name']);
if ($imageInfo === false) {
    $errors[] = 'File is not a valid image.';
}
| Ctrl+L to chat, Ctrl+K to generate
return ['valid' => empty($errors), 'errors' => $errors, 'extension' => $extension, 'mimeType' => $mimeType];
```

2.6 Secure Filename Generation:

```
// Generate secure filename: timestamp_userid_randomhash.extension
$extension = $validation['extension'];
$randomHash = bin2hex(random_bytes(16));
$filename = time() . '_' . $userId . '_' . $randomHash . '.' . $extension;
$filepath = UPLOAD_DIR . $filename;
```

2.7 Directory Security (.htaccess):

```
# Security: Prevent PHP execution in uploads directory
# This is a critical security measure to prevent uploaded malicious files from being executed

# Disable PHP execution
<FilesMatch "\.php$">
    Order Deny,Allow
    Deny from all
</FilesMatch>

# Alternative method for Apache 2.4+
<IfModule mod_php.c>
    php_flag engine off
</IfModule>
| Ctrl+L to chat, Ctrl+K to generate
# Disable script execution
Options -ExecCGI
AddHandler cgi-script .php .php3 .php4 .php5 .phtml .pl .py .jsp .asp .aspx .sh .cgi

# Force files to be downloaded instead of executed
<FilesMatch "\.(php|php3|php4|php5|phtml|pl|py|jsp|asp|aspx|sh|cgi)$">
    ForceType application/octet-stream
    Header set Content-Disposition attachment
</FilesMatch>

# Only allow image files to be accessed
<FilesMatch "^([^\.]+\.(jpg|jpeg|png|gif|webp))$">
    Order Allow,Deny
    Allow from all
</FilesMatch>

# Deny access to .htaccess itself
<Files .htaccess>
```

```
# Deny access to .htaccess itself
<Files .htaccess>
|   Order Allow,Deny
|   Deny from all
|
</Files>
```

3.XSS Prevention:

```
<div class="detail-section">
|   <div class="section-label">Subject</div>
|   <div class="section-content subject">
|       <?php echo htmlspecialchars($request['subject']); ?>
|   </div>
</div>

<div class="detail-section">
|   <div class="section-label">Description</div>
|   <div class="section-content description">
|       <?php echo htmlspecialchars($request['description']); ?>
|   </div>
</div>
```

Task 5 - Admin Request List

The Admin Request List feature provides administrators with a centralized interface to manage all customer evaluation requests.

Only authenticated administrators can access this area. The system implements role-based access control, status filtering, and secure status updates with full CSRF protection.

Why this admin page is secure

1. Access Control

1.1 Login Check:

```
// 1) Must be logged in
if (!isset($_SESSION['user_id'])) {
    header('Location: ../login.php');
    exit;
}
```

1.2 Admin Role Check (Authorization):

```
// 2) Must be an administrator
if (empty($_SESSION['user_is_admin']) || $_SESSION['user_is_admin'] != 1) {
    header('Location: ../dashboard.php');
    exit;
}
```

2. Status Update Security

2.1 POST Method Required:

```
// 3) Only accept POST requests
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    header('Location: requests.php');
    exit;
}
```

2.2 CSRF Protection:

```
// 4) CSRF Protection
if (!verifyCSRFToken()) {
    $_SESSION['flash_error'] = 'Invalid security token. Please try again.';
    header('Location: requests.php');
    exit;
}
```

2.3 Status Value Whitelist Validation:

```
// Status label mapping (must match requests.php)
$allowedStatuses = [
    'pending',
    'open',
    'closed',
    'in_progress',
    'resolved',
];
```

```
// Validate status
if (!in_array($newStatus, $allowedStatuses)) {
    $_SESSION['flash_error'] = 'Invalid status value.';
    header('Location: requests.php');
    exit;
}
```

3. SQL Injection Prevention

3.1 Prepared Statements for Query:

```
// Build query based on filter
if ($filterStatus && in_array($filterStatus, array_keys($statusLabels))) {
    $stmt = $pdo->prepare(
        'SELECT r.id, r.subject, r.description, r.contact_preference, r.photo_path, r.status, r.created_at, r.updated_at,
        u.id as user_id, u.full_name as user_name, u.email as user_email
        FROM requests r
        LEFT JOIN users u ON r.user_id = u.id
        WHERE r.status = ?
        ORDER BY r.created_at DESC'
    );
    $stmt->execute([$filterStatus]);
}
```

3.2 Prepared Statements for Update

```
$stmt = $pdo->prepare(
    'UPDATE requests SET status = ?, updated_at = NOW() WHERE id = ?'
);
$stmt->execute([$newStatus, $requestId]);
```

4. XSS Prevention:

```
<div class="request-subject"><?php echo htmlspecialchars($request['subject']); ?></div>
<div class="request-user">
    <strong>User:</strong> <?php echo htmlspecialchars($userName); ?>
    <?php Convert special characters to HTML entities
</div>
</div>
<div>
    htmlspecialchars( string $string [, int $flags = ENT_COMPAT | ENT_HTML401 [, string
    $encoding = ini_get("default_charset") [, bool $double_encode ]]): string
<span class="re
    <?php echo htmlspecialchars($statusLabel); ?>
```

5. Request ID Validation

```
$requestId = isset($_POST['request_id']) ? (int)$_POST['request_id'] : 0;
$newStatus = isset($_POST['status']) ? trim($_POST['status']) : '';

// Validate request ID
if ($requestId <= 0) {
    $_SESSION['flash_error'] = 'Invalid request ID.';
    header('Location: requests.php');
    exit;
}
```

Task 6 - Database Design

The database design follows security best practices with proper data types, password hashing storage, token-based verification systems, and relational integrity. The schema supports all security features including email verification, 2FA, account lockout, and secure password recovery.

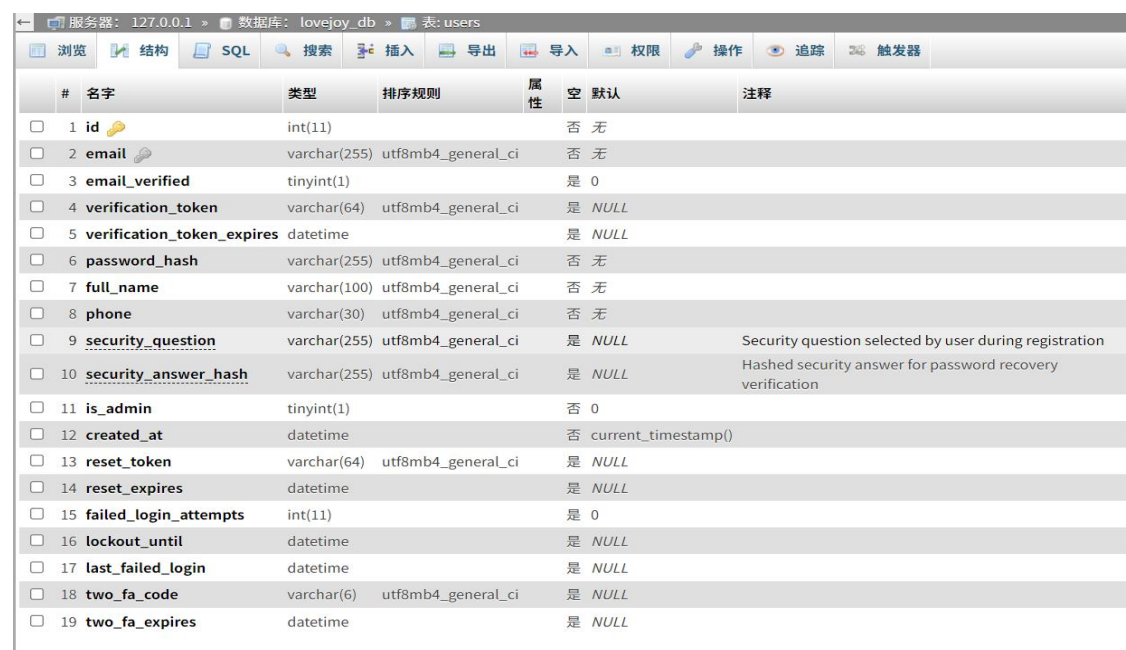
1. Database Schema Overview



表	操作	行数	类型	排序规则	大小	多余
☐ password_resets		0	InnoDB	utf8mb4_general_ci	48.0 KB	-
☐ requests		0	InnoDB	utf8mb4_general_ci	48.0 KB	-
☐ users		2	InnoDB	utf8mb4_general_ci	48.0 KB	-
3 张表	总计	2	InnoDB	utf8mb4_general_ci	144.0 KB	0 字节

2. Table Structures

2.1 Users Table (19 fields)



#	名字	类型	排序规则	属性	空	默认	注释
☐ 1	id	int(11)		否	无		
☐ 2	email	varchar(255)	utf8mb4_general_ci	否	无		
☐ 3	email_verified	tinyint(1)		是	0		
☐ 4	verification_token	varchar(64)	utf8mb4_general_ci	是	NULL		
☐ 5	verification_token_expires	datetime		是	NULL		
☐ 6	password_hash	varchar(255)	utf8mb4_general_ci	否	无		
☐ 7	full_name	varchar(100)	utf8mb4_general_ci	否	无		
☐ 8	phone	varchar(30)	utf8mb4_general_ci	否	无		
☐ 9	security_question	varchar(255)	utf8mb4_general_ci	是	NULL		Security question selected by user during registration
☐ 10	security_answer_hash	varchar(255)	utf8mb4_general_ci	是	NULL		Hashed security answer for password recovery verification
☐ 11	is_admin	tinyint(1)		否	0		
☐ 12	created_at	datetime		否	current_timestamp()		
☐ 13	reset_token	varchar(64)	utf8mb4_general_ci	是	NULL		
☐ 14	reset_expires	datetime		是	NULL		
☐ 15	failed_login_attempts	int(11)		是	0		
☐ 16	lockout_until	datetime		是	NULL		
☐ 17	last_failed_login	datetime		是	NULL		
☐ 18	two_fa_code	varchar(6)	utf8mb4_general_ci	是	NULL		
☐ 19	two_fa_expires	datetime		是	NULL		

2.2 Requests Table (9 fields)

← 服务器: 127.0.0.1 » 数据库: lovejoy_db » 表: requests

浏览 结构 SQL 搜索 插入 导出 导入 权限 操作 追踪 触发器

表结构 关联视图

#	名字	类型	排序规则	属性	空	默认	注释	额外	操作
☐	1 id 🔑	int(11)			否	无		AUTO_INCREMENT	修改 删除 更多
☐	2 user_id 🔑	int(11)			否	无			修改 删除 更多
☐	3 subject	varchar(255)	utf8mb4_general_ci		否	无			修改 删除 更多
☐	4 description	text	utf8mb4_general_ci		否	无			修改 删除 更多
☐	5 contact_preference	varchar(10)	utf8mb4_general_ci	是		email			修改 删除 更多
☐	6 photo_path	varchar(255)	utf8mb4_general_ci	是		NULL			修改 删除 更多
☐	7 status	varchar(50)	utf8mb4_general_ci		否	pending			修改 删除 更多
☐	8 created_at	datetime			否	current_timestamp()			修改 删除 更多
☐	9 updated_at	datetime			否	current_timestamp()	ON UPDATE CURRENT_TIMESTAMP()		修改 删除 更多

2.3 Password Resets Table (5 fields)

← 服务器: 127.0.0.1 » 数据库: lovejoy_db » 表: password_resets

浏览 结构 SQL 搜索 插入 导出 导入 权限 操作 追踪 触发器

表结构 关联视图

#	名字	类型	排序规则	属性	空	默认	注释	额外	操作
☐	1 id 🔑	int(11)			否	无		AUTO_INCREMENT	修改 删除 更多
☐	2 user_id 🔑	int(11)			否	无			修改 删除 更多
☐	3 token 🔑	varchar(255)	utf8mb4_general_ci		否	无			修改 删除 更多
☐	4 expires_at	datetime			否	current_timestamp()			修改 删除 更多
☐	5 created_at	datetime			否	current_timestamp()			修改 删除 更多